

Report tecnico – Confronto `.pt` vs `.hef` per Plate Detection

1. Obiettivo

Questo report confronta le prestazioni dei due modelli YOLOv11n per **license plate detection** prima e dopo la conversione in formato `.hef`, necessario per l'esecuzione su Raspberry Pi + Hailo-8 AI HAT+.

Il confronto è utile perché durante la conversione `ONNX` → `HEF` la fase di ottimizzazione Hailo è stata eseguita **senza GPU**, con riduzione dell'optimization level a 0. L'obiettivo è quindi verificare se la conversione abbia causato una perdita significativa di accuratezza.

I confronti sono:

- `yolov11nUC3M.pt` vs `yolov11n_uc3m.hef`
- `license-plate-finetune-v1n.pt` vs `license_plate_finetune_v1n.hef`

2. Dataset e metodologia

Il dataset utilizzato è lo stesso per i test `.pt` e `.hef`.

Elemento	Dettaglio
Dataset	Car Plate Detection YOLOv8
Link	https://www.kaggle.com/datasets/nimapourmoradi/car-plate-detection-yolov8
Numero immagini	1381
Ground truth bounding box	1474
Classe	Plates
Task	License Plate Detection

I modelli `.pt` sono stati valutati su Colab tramite Ultralytics YOLO.

I modelli `.hef` sono stati valutati direttamente su Raspberry Pi 5 con Hailo-8 AI HAT+ tramite HailoRT.

Nel benchmark HEF sono stati usati IoU = 0.5, letterboxing con preservazione aspect ratio, riconversione delle coordinate nello spazio originale e NMS integrato per i modelli YOLOv11n.

3. Risultati complessivi

Modello	Formato	Precision	Recall	F1-score	mAP@50	mAP@50-95	TP	FP	FN	FPS
UC3M	.pt	0.9207	0.9534	0.9368	0.9748	0.6556	1462	323	12	n.d.
UC3M	.hef	0.7490	0.9800	0.8490	0.9560	n.d.	1445	485	29	91.1
v1n	.pt	0.9856	0.9775	0.9816	0.9936	0.5487	1464	60	10	n.d.
v1n	.hef	0.9170	0.9980	0.9555	0.9940	n.d.	1471	134	3	108.8

I dati `.hef` derivano dal benchmark Hailo-8 precedente, in cui `license_plate_finetune_v1n.hef` ottiene F1 = 0.9555 e `yolov11n_uc3m.hef` ottiene F1 = 0.8490.

4. Confronto UC3M .pt vs UC3M .hef

Metrica	UC3M .pt	UC3M .hef	Differenza
Precision	0.9207	0.7490	-0.1717
Recall	0.9534	0.9800	+0.0266
F1-score	0.9368	0.8490	-0.0878
mAP@50	0.9748	0.9560	-0.0188
TP	1462	1445	-17
FP	323	485	+162
FN	12	29	+17

Analisi

La conversione del modello UC3M in `.hef` porta a un peggioramento evidente.

Il calo principale riguarda la **precision**, che passa da 0.9207 a 0.7490. Questo significa che il modello convertito produce molte più detection errate. Il dato è confermato dall'aumento dei falsi positivi: da 323 a 485, cioè 162 falsi positivi in più.

Anche i falsi negativi aumentano, passando da 12 a 29. Quindi il modello `.hef` non solo genera più falsi positivi, ma perde anche più targhe rispetto alla versione `.pt`.

La recall risulta più alta nella versione `.hef`, ma questo valore va interpretato con cautela, perché i conteggi assoluti mostrano un aumento dei falsi negativi. La differenza può dipendere da soglie, NMS o metodo di calcolo diverso tra Ultralytics e HailoRT.

Conclusione UC3M: la conversione è tecnicamente riuscita, ma causa una perdita rilevante di qualità, soprattutto per l'aumento dei falsi positivi.

5. Confronto v1n `.pt` vs v1n `.hef`

Metrica	v1n <code>.pt</code>	v1n <code>.hef</code>	Differenza
Precision	0.9856	0.9170	-0.0686
Recall	0.9775	0.9980	+0.0205
F1-score	0.9816	0.9555	-0.0261
mAP@50	0.9936	0.9940	+0.0004
TP	1464	1471	+7
FP	60	134	+74
FN	10	3	-7

Analisi

Il modello v1n mantiene prestazioni molto alte anche dopo la conversione in `.hef`.

La precision diminuisce da 0.9856 a 0.9170, quindi anche in questo caso la versione `.hef` genera più falsi positivi. I FP passano infatti da 60 a 134.

Tuttavia, il degrado è molto più contenuto rispetto a UC3M. L’F1-score passa da 0.9816 a 0.9555, con una perdita di soli 0.0261 punti. Inoltre, la recall aumenta e i falsi negativi diminuiscono da 10 a 3. Questo significa che la versione `.hef` del modello v1n perde pochissime targhe.

Il mAP@50 rimane praticamente invariato: 0.9936 per il `.pt` e 0.9940 per il `.hef`.

Conclusione v1n: la conversione è riuscita bene. Il modello perde un po’ di precision, ma mantiene un livello di accuratezza molto elevato ed è il candidato migliore per la pipeline.

6. Impatto della conversione

Perdita di F1-score

Modello	F1 .pt	F1 .hef	Differenza	Perdita relativa
UC3M	0.9368	0.8490	-0.0878	-9.37%
v1n	0.9816	0.9555	-0.0261	-2.66%

Aumento dei falsi positivi

Modello	FP .pt	FP .hef	Differenza
UC3M	323	485	+162
v1n	60	134	+74

Variazione dei falsi negativi

Modello	FN .pt	FN .hef	Differenza
UC3M	12	29	+17
v1n	10	3	-7

L'effetto principale della conversione è l'aumento dei falsi positivi. Questo vale per entrambi i modelli, ma l'impatto è molto più forte su UC3M.

Il modello v1n, invece, mantiene una qualità complessiva elevata e nella versione .hef riduce anche il numero di targhe perse.

7. Conclusione

Il confronto mostra che la conversione ONNX → HEF ha un impatto misurabile sulle prestazioni, soprattutto sulla precision e sul numero di falsi positivi.

Per il modello **UC3M**, la perdita è significativa:

Aspetto	Risultato
F1-score	-0.0878

Aspetto	Risultato
Precision	-0.1717
False Positives	+162
False Negatives	+17

Per il modello **v1n**, la conversione è molto più stabile:

Aspetto	Risultato
F1-score	-0.0261
Precision	-0.0686
False Positives	+74
False Negatives	-7
mAP@50	Praticamente invariato

Il modello consigliato per la pipeline NGS2026 è quindi:

license_plate_finetune_v1n.hef

Rispetto a UC3M, il modello v1n **.hef** ha F1-score più alto, precision migliore, recall quasi perfetta, meno falsi positivi e solo 3 falsi negativi su 1474 bounding box. Inoltre raggiunge 108.8 FPS su Hailo-8, valore ampiamente sufficiente per l'utilizzo nella pipeline real-time.

La conversione del modello v1n può quindi essere considerata ben riuscita, nonostante l'ottimizzazione sia stata eseguita senza GPU. UC3M **.hef** rimane utilizzabile, ma non rappresenta la scelta migliore per il deployment finale.